Bound proof

Heuristic. Choose earliest drop-off.

Problem statement (simplified)

Let G be an $N \times N$ Manhattan grid with intersections $(x, y) \in \{0, \dots, N\}^2$ and distance $d(a, b) = |x_a - x_b| + |y_a - y_b|$. Travelling one block takes one unit of time. There are m identical cars, all at a depot $o \in G$ at time 0, and each car carries one passenger at a time.

We are given n trips $r_i = (s_i, w_i, t_i)$ for $i = 1, \dots, n$, where s_i is the pickup point, w_i the drop-off point and $t_i \geq 0$ the request time. All trips are known in advance.

A car serves a trip by leaving the depot, driving to s_i, carrying the passenger to w_i, and driving back to o. It must be back at the depot before it starts another trip, and it cannot leave for trip i before time t_i. So if trip i is started at time $S_i \geq t_i$, the car is busy for

$$p_i = d(o, s_i) + d(s_i, w_i) + d(w_i, o),$$

and trips assigned to the same car must not overlap in time.

The passenger of trip i is dropped off at $D_i = S_i + d(o, s_i) + d(s_i, w_i)$, and their completion time is $F_i = D_i - t_i$. The goal is to assign every trip to a car and choose start times S_i so as to minimise the maximum completion time

$$F_{\max} = \max_{1 \leq i \leq n} (D_i - t_i).$$

Theorem. Consider the heuristic that chooses the trip with the earliest possible drop off, chosen across all car-trip pairs.
Proof. Let O be the assignment produced by the optimal algorithm, and let G be the greedy assignment. For each trip k, let $a_k = d(o, s_k) + d(s_k, w_k)$ and $p_k = a_k + d(w_k, o)$.

Claim. G always produces an assignment with cost at most $\left(6 - \frac{1}{m}\right) \mathrm{OPT}$.

Lower bounds. Since every trip k has travel time at least a_k, we have $a_k \leq \mathrm{OPT}$ for every trip k. Further, the processing time of trip k is bounded by $p_k \leq 2\,\mathrm{OPT}$, since $d(w_k, o) \leq a_k$ by the triangle inequality, so $p_k \leq 2a_k$.

Proof. Now consider the worst trip, the trip with the largest completion time; call it j. Its cost is

$$G_j = (S_j - t_j) + a_j,$$

i.e. waiting plus its own trip time, where $a_j \leq \mathrm{OPT}$.
If j was chosen exactly when it was requested, its cost is $a_j \leq \mathrm{OPT}$.
If j was waiting because every car was busy, go back to t_0, the last moment before S_j when some car was idle or a car took a trip with a larger key than j. For the algorithm G, m cars are busy for $S_j - t_0$, so the total work done is $m(S_j - t_0)$. O must do the same trips but within a limited window, which caps the total busy time of those trips. We compare the two bounds to see how long j waited.

Trips in progress at t_0. There are at most m of them, and each has at most $p_k \le 2\,\mathrm{OPT}$ of driving left, so they contribute at most $2m\,\mathrm{OPT}$.
Trips taken after t_0 and before S_j.

- Earliest requested: t_0, since otherwise a car would have taken it at t_0.
- Latest requested: since we use G's rule, a trip i is only chosen before j if its earliest possible drop-off is no later than j's, i.e. $t_i + a_i \le t_j + a_j$. So $t_i \le t_j + a_j \le t_j + \mathrm{OPT}$.

In O, trip i's car leaves after $t_i > t_0$ and is back by $t_i + \mathrm{OPT} + d(w_i, o) \le t_j + 3\,\mathrm{OPT}$. So in O all of these trips fit inside the window $[t_0,\, t_j + 3\,\mathrm{OPT}]$, and m cars can do at most $m(t_j + 3\,\mathrm{OPT} - t_0)$ driving in it. Trip j itself is not driven before S_j, so

$$m(S_j - t_0) \le 2m\,\mathrm{OPT} + m(3\,\mathrm{OPT} + t_j - t_0) - p_j \quad \Longrightarrow \quad S_j - t_j \le 5\,\mathrm{OPT} - \frac{p_j}{m}.$$

Using $p_j \ge a_j$ and $a_j \le \mathrm{OPT}$,

$$G_j = (S_j - t_j) + a_j \le 5\,\mathrm{OPT} + a_j - \frac{a_j}{m} = 5\,\mathrm{OPT} + \left(1 - \frac{1}{m}\right) a_j \le \left(6 - \frac{1}{m}\right) \mathrm{OPT}.$$

□